

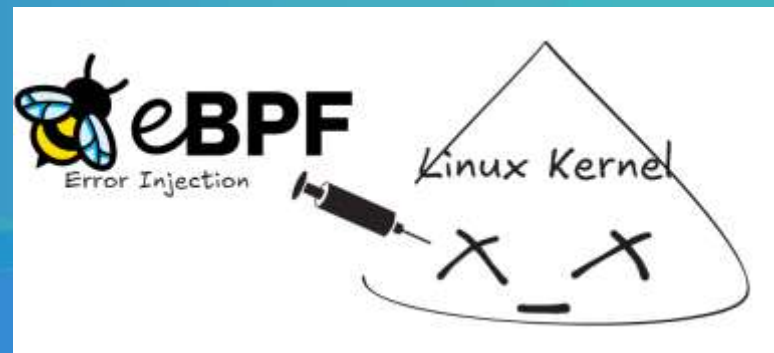


第三届 eBPF开发者大会

www.ebpftravel.com

基于eBPF和LLM样例推荐错误注入测试系统

西安邮电大学 大四 刘佳欢 赵嘉豪



中国·西安

1. 什么是错误注入

硬件错误注入

通过物理手段干扰硬件运行条件（如电压、时钟频率、电磁场等），强制芯片或电路进入异常状态。

软件错误注入

修改源代码或二进制代码，模拟错误发生。

内核错误注入

内核的异常处理代码路径往往难以通过常规测试触发。通过故障注入函数，可强制触发如磁盘 I/O 错误 (`should_fail_bio`)、网络丢包 (`should_fail_skb`) 等场景，确保这些冷门但关键的代码路径被覆盖。

2. 内核原生错误注入框架

内核原生错误注入框架是Linux内核机制，用于在运行时模拟硬件故障、软件异常等场景，验证系统的容错能力与错误处理逻辑。

其核心目标是：

- 提升内核代码的健壮性；
- 加速异常场景的复现与调试

主要技术分支：

Fault-injection：内核原生支持的**静态注入**框架，覆盖内存分配失败、磁盘IO错误等场景。

KPROBE_OVERRIDE：基于eBPF的**动态注入**机制，允许运行时覆盖函数返回值，灵活性更高

2.1 技术分支-Fault-Injection框架

支持的注入类型：

- **内存错误**: fail_page_alloc (页分配失败)、failslab (slab分配失败) ;
- **磁盘IO错误**: fail_make_request (模拟块设备IO错误)、fail_io_timeout (IO超时)
-
- **以及其他子系统手动实现的错误注入函数**

配置

运行时通过debugfs动态调整触发概率、间隔等参数

如：

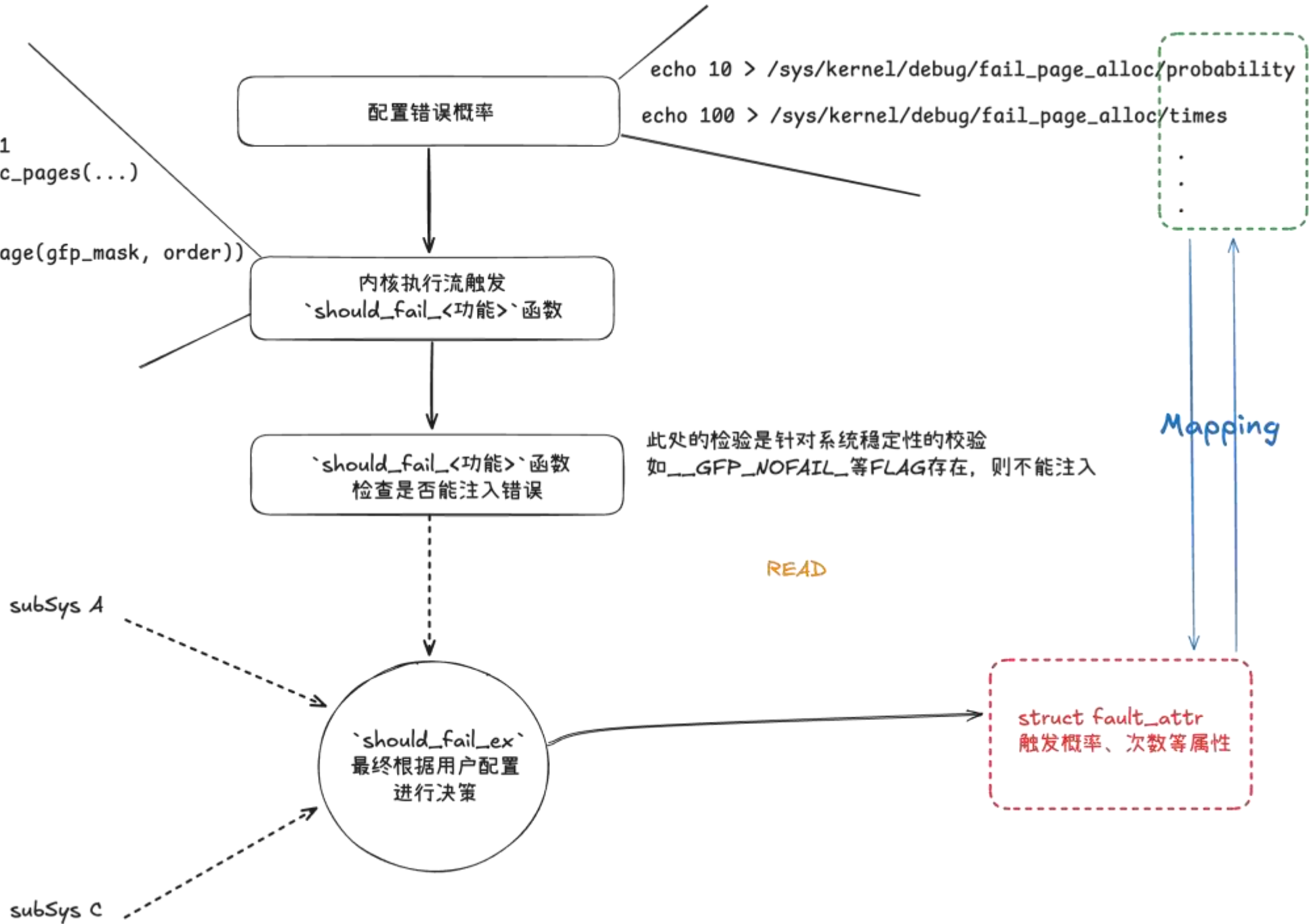
```
echo 10 >
```

```
/sys/kernel/debug/fail_page_alloc/probability
```

```
echo 100 > /sys/kernel/debug/fail_page_alloc/times
```

上述命令会使内存分配有 10% 的概率失败，最多触发 100 次。

```
//v6.13.7 /mm/page_alloc.c#L4491
static inline bool prepare_alloc_pages(...)
{
    ...
    if (should_fail_alloc_page(gfp_mask, order))
        return false;
    ...
    return true;
}
```



Defined in 2 files as a member:

drivers/nvme/host/nvme.h, line 292 *(as a member)*drivers/nvme/host/nvme.h, line 388 *(as a member)*

Referenced in 3 files:

drivers/nvme/host/core.c

- line 3623
- line 3654
- line 4070
- line 4170

drivers/nvme/host/fault_inject.c

- line 51
- line 54
- line 60
- line 67
- line 67
- line 71
- line 71
- line 74
- line 74
- line 76
- line 77

drivers/nvme/host/nvme.h, line 412

should_fail

should_fail**should_fail_alloc_page****should_fail_bad_hdr****should_fail_bad_hdr_ebadmsg****should_fail_bio****should_fail_bitflips****should_fail_eccerr****should_fail_erase_failure****should_fail_ex****should_fail_futex**


```
bool should_fail_alloc_page(gfp_t gfp_mask, unsigned int order) {
    int flags = 0;

    // 条件1: 检查分配阶是否低于阈值
    if (order < fail_page_alloc.min_order)
        return false;

    // 条件2: 检查是否设置了 __GFP_NOFAIL 标志 (禁止失败)
    if (gfp_mask & __GFP_NOFAIL)
        return false;

    // 条件3: 若配置忽略 HIGHMEM 区域, 则跳过相关分配请求
    if (fail_page_alloc.ignore_gfp_highmem && (gfp_mask & __GFP_HIGHMEM))
        return false;

    // 条件4: 若配置忽略直接回收请求, 则跳过相关分配
    if (fail_page_alloc.ignore_gfp_reclaim && (gfp_mask & __GFP_DIRECT_RECLAIM))
        return false;

    // 设置标志: 若分配请求要求不输出警告信息 (__GFP_NOWARN)
    if (gfp_mask & __GFP_NOWARN)
        flags |= FAULT_NOWARN;

    // 最终决策: 调用错误注入框架判断是否触发失败
    return should_fail_ex(&fail_page_alloc.attr, 1 << order, flags);
}
```

`should_fail_alloc_page`
in /mm/fail_page_alloc

```
/*
 * For explanation of the elements of this struct, see
 * Documentation/fault-injection/fault-injection.rst
 */
struct fault_attr {
    unsigned long probability;
    unsigned long interval;
    atomic_t times;
    atomic_t space;
    unsigned long verbose;
    bool task_filter;
    unsigned long stacktrace_depth;
    unsigned long require_start;
    unsigned long require_end;
    unsigned long reject_start;
    unsigned long reject_end;

    unsigned long count;
    struct ratelimit_state ratelimit_state;
    struct dentry *dname;
};
```

错误触发的属性 具体结构

2.2 技术分枝-(eBPF)动态错误注入

基于eBPF的**bpf_override_return()**函数，劫持函数执行流并覆盖返回值

流程

- 注册kprobe探测目标函数
- 通过eBPF程序调用 `bpf_override_return()`，将程序计数器(PC) 重定向至替代函数
- 跳过原函数执行，直接返回预设错误码

配置

- 内核编译时启用

`CONFIG_BPF_KPROBE_OVERRIDE`选项；

- 编写eBPF程序绑定目标函数，注入错误返回值（如-EIO）



User space

kernel space

eBPF prog:

...
SEC("kprobe/XXX_XXX_XX")

`bpf_override_return(args)`

BPF SYSCALL

eBPF
verifier

within_error_injection_list?
if def CONFIG_BPF_KPROBE_OVERRIDE?

bpf_helpers

bpf_override_return_proto

regs_set_return_value

override_function_with_return

(void *) 58

函数返回值覆盖

```
static inline void regs_set_return_value(struct pt_regs *regs, unsigned long rc)
{
    regs->ax = rc;
}
```

/arch/x86/include/asm/ptrace.h

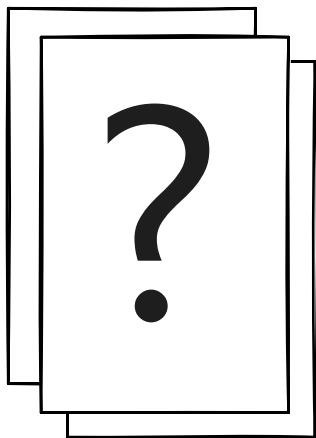
```
void override_function_with_return(struct pt_regs *regs)
{
    regs->ip = (unsigned long)&just_return_func;
}
```

```
NOKPROBE_SYMBOL(override_function_with_return);
```

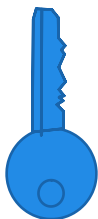
/arch/x86/lib/error_inject.c

原生限制与白名单机制

_error_injection_whitelist



within_error_injection_list?



需要针对目标函数在内核源码打上宏
`ALLOW_ERROR_INJECTION(fname, _etype)`

The Linux kernel configuration item `CONFIG_BPF_KPROBE_OVERRIDE` :

- prompt: Enable BPF programs to override a kprobed function
- type: bool
- depends on: `CONFIG_BPF_EVENTS` and `CONFIG_FUNCTION_ERROR_INJECTION`
- defined in `kernel/trace/Kconfig`
- found in Linux kernels: 4.16–4.20, 5.0–5.19, 6.0–6.10, 6.11-rc+HEAD

```
// 这里可能需要 #include <asm-generic/error-injection.h>
ALLOW_ERROR_INJECTION(mlx5_cmd_exec, ERRNO);

int mlx5_cmd_exec(struct mlx5_core_dev *dev, void *in, int in_size, void *out,
                  int out_size)
{
    int err;

    err = cmd_exec(dev, in, in_size, out, out_size, NULL, NULL, false);
    return err ? : mlx5_cmd_check(dev, in, out);
}
```

错误白名单注册

```
#define ALLOW_ERROR_INJECTION(fname, _etype)

static struct error_injection_entry __used
    __section("_error_injection_whitelist")
    _eil_addr_##fname = {
    .addr = (unsigned long)fname,
    .etype = EI_ETYPE_##_etype,
};
```

内核在启动时通过链接脚本 (linker script) 收集所有位于 `_error_injection_whitelist` 段的结构体实例，形成全局白名单链表。

`error_injection_entry` 结构体：表示一个错误注入白名单条目，包含两个字段：

addr：函数地址，通过 `(unsigned long)fname` 将函数名转换为地址。

etype：错误类型，由 `EI_ETYPE_##_etype` 生成，例如 `EI_ETYPE_NULL` 表示空指针错误

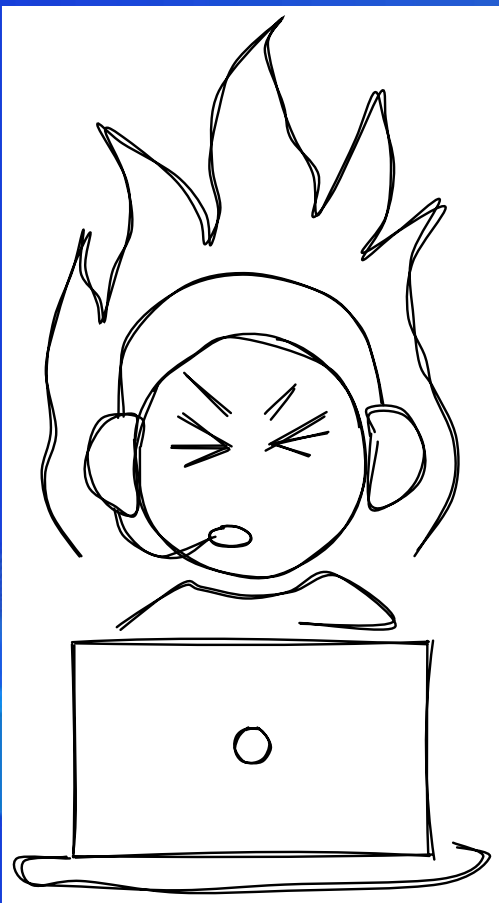
[vmlinux.lds.h](#)

```
#ifdef CONFIG_FUNCTION_ERROR_INJECTION
#define ERROR_INJECT_WHITELIST()    STRUCT_ALIGN();
    __start_error_injection_whitelist = .;
    KEEP(*(_error_injection_whitelist))
    __stop_error_injection_whitelist = .;
#else
#define ERROR_INJECT_WHITELIST()
#endif
```

3. 我做了什么?

对比维度	实现方式	侵入性	灵活性	性能开销	安全性	调试能力	兼容性	限制性
静态错误注入框架	修改内核代码	高	低	低	弱	弱	一般	无, 可任意修改逻辑
eBPF动态错误注入手段	kprobehook内核函数并覆盖行为	低	高	中	可控	强	较高	有, 受限于eBPF verifier

深入思考



在大模型检索、生成能力大幅度增强的大背景下，我们是否能在两个框架上取长补短，做些什么？回顾错误注入的场景：

自研驱动测试：

驱动编写完成->编写测试用例->冒烟/故障注入测试

CVE/Bugs的分布：

相当多的因未处理好返回值而导致的权限提升、执行流劫持等CVE。

需要模拟内核故障的场景：

选定子系统和子系统的具体函数->编写基于现有错误注入框架下的代码->重新编译内核，如果策略变更需要再次重复过程。

部分需要模拟硬件故障的场景：

复现困难、条件苛刻。

预期

- 能否在测试环境下，不在频繁打ALLOW_INJECTION宏，且不再重新编译内核？
- 能否利用大模型，将用户描述的错误发生的场景自动的模拟出来，并生成eBPF程序进行错误注入？
- 能否利用大模型，自动的去模拟任意子系统中允许范围内的错误，类似fuzzing的形式去触发error？

3.1 魔改内核

Q: 我们有没有方法绕过?

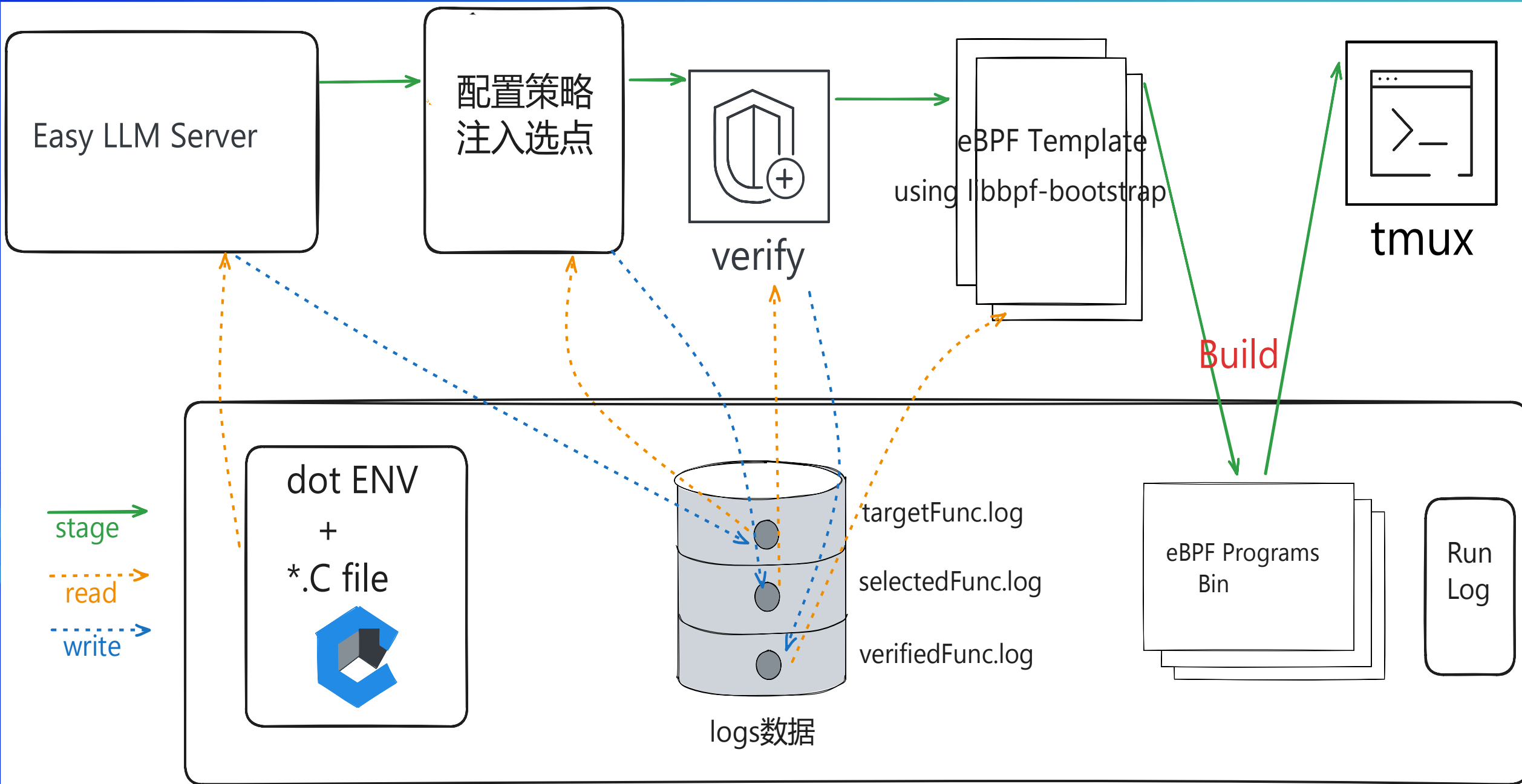
```
/kernel/bpf/verifier.c
int bpf_check_attach_target(...)
{
    ...
    if(within_error_injection_list(addr))
    ...
}
```

A: 绕过方案: 加一个CONFIG

```
bool within_error_injection_list(unsigned long addr)
{
    // struct ei_entry *ent;
    // bool ret = false;

    // mutex_lock(&ei_mutex);
    // list_for_each_entry(ent, &error_injection_list, list) {
    //     if (addr >= ent->start_addr && addr < ent->end_addr) {
    //         ret = true;
    //         break;
    //     }
    // }
    // mutex_unlock(&ei_mutex);
    // return ret;
    return true;
}
```

3.2 系统结构



选点机制

1. 用户手动选择错误注入函数+概率
2. 大模型根据源码推荐进行错误注入的函数
3. 根据用户描述场景，返回错误注入目标函数及错误返回值

过滤机制

1. 校验改函数是否存在于/proc/kallsyms（即，当前内核是否有该函数）
2. 提供一组错误宏，限定错误智能发生在-EIO,-EBUSY,-ENOMEM...等,防止预期外的错误
3. prompt增强

注入策略

1. 大模型推荐注入点+单错误值概率、多错误值概率、默认概率配置
2. 手动选点+配置概率
3. 用户自定义触发策略（利用eBPF特性）

运行策略

手工注入

自动注入-渐进注入

自动注入-批量注入

反馈

1. 转发trace_pipe到dmesg
2. 运行状态定时检查, 程序执行退出通知, 使用tmux复用终端

```
e_install_pte...
2024-12-31 14:44:32 - Preloading ../build/bin/migrate_vma
_collect_pnd...
2024-12-31 14:44:32 - Preloading ../build/bin/mincore_pte
_range...
2024-12-31 14:44:32 - Preloading ../build/bin/min_ratio_s
tore...
2024-12-31 14:44:32 - Preloading ../build/bin/mlock_folio
...
6): apparmor="DENIED" operation="open" class="file" profile="snap-update-ns.firmware-updater" name="/proc/645939/
/maps" pid=645939 comm="5" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[410749.455839] audit: type=1400 audit(1735617635.771:29
7): apparmor="DENIED" operation="open" class="file" profile="snap.firmware-updater.firmware-notifier" name="/proc/645922/c/sys/vm/max_map_count" pid=645922 comm="firmware-notifier" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[faulTinje@:bash* "server-Default-string" 14:44 31-12月-24
```

```
Function create_cache not found in /proc/kallsyms, skipping.
Function cache_alloc_handle not found in /proc/kallsyms, skipping.
Function cache_alloc_zspage not found in /proc/kallsyms, skipping.
过滤完成, 结果已保存到 ../logs/verifiedFunc.log
```

3.3 使用方式

0. 在仓库根目录创建 `.env` 文件,并根据实际情况填写。

```
OPENAI_API_KEY=xxxxxx  
OPENAI_BASE_URL=xxxxx
```



1. 启动LLM Server:

```
faultInjectionLLM server
```

2. 添加需要分析的注入点所在源文件 (支持手工添加注入点):

```
faultInjectionLLM add [default | multi | manual] <Absolute-Path-Of-SourceCode-File>
```

3. 根据LLM的推荐具体选择注入点以及概率:

```
faultInjectionLLM choose [-i | default]
```

4. 验证

```
faultInjectionLLM verify
```

5. 生成注入执行程序

```
faultInjectionLLM gen
```

6. 执行错误注入程序:

```
faultInjectionLLM tmux [once | progressive]
```

3. 参数解释

- add
 - **default**: 默认使用所有的LLM推荐的错误注入点以及错误类型，每个类型被随机触发的概率为5%。
 - **multi**: 使用所有的LLM推荐的错误注入点以及错误类型。
 - **manual**: 手动选择额外的错误注入点以及错误类型，并可以自定义每个概率的触发概率
- choose
 - **-i**: 交互式选择错误注入值以及每个值的概率
 - **default**: 默认（函数的所有错误值都会被触发，都为5%的概率，函数被触发错误的概率为 $N \times 0.05$, 不超过1)
- tmux
 - **once**: 一次性使能所有错误注入点。
 - **progressive**: 每五秒加载一个错误注入点，同时不会覆盖已加载的错误注入点。

4. 其他使用方法

1. 查看已添加的注入点:

```
faultInjectionLLM list [recommend | select | verified]
```

2. 清除历史文件:

```
faultInjectionLLM clean
```

4. 测试

测试 kmalloc错误注入

(内核原生已支持)

测试 xx子系统错误注入

内核原生不支持

测试 xxx 自研硬件驱动错误注入

模拟物理机/代码缺陷问题

下一步是？

1. 集成为MCP Server，不再需要用户输入源文件，拓展更多协作可能性。
2. 重构。
3. 持续探索在fuzzing、漏洞挖掘方面的可能性。
4. 支持更多内置策略。

Q&A

感谢您的聆听！

欢迎加入本项目的开发

https://github.com/JiaHuann/Smart_Fault_Injector_LLM